

BezBroker

ЧАСТ 2 ОТ 2

Част 2: Разработка на мобилното приложение

Java + Navigation Drawer + RecyclerView + osmdroid. Линейно — всеки урок добавя само това, което му трябва.

Съдържание

1. Урок 01 — Преглед
2. Урок 02 — Списък с имоти
3. Урок 03 — Детайли на имот
4. Урок 04 — Добавяне на имот
5. Урок 05 — Карта с имоти (osmdroid)
6. Урок 06 — Оферти
7. Урок 07 — Чат
8. Урок 08 — Logout

Урок 01 — Преглед

Стартова точка

В проекта вече има:

- `LoginActivity` и `RegisterActivity` — пращат заявки към API-то.
- `ApiClient` — собствен HTTP клиент с методи `post(...)` и `get(...)`.
- `SessionManager` — пази токен + потребител в `SharedPreferences`.
- `MainActivity` — `Navigation Drawer` шаблон от `Android Studio` с три `placeholder` фрагмента: `HomeFragment`, `GalleryFragment`, `SlideshowFragment`.

Не пипаме нищо от тези файлове в Урок 01.

Какво ще построим

№	Урок	Какво се случва
02	Списък с имоти	Подменяме <code>HomeFragment</code> с <code>EstatesFragment</code> . Добавяме библиотеки <code>RecyclerView</code> , <code>Glide</code> , <code>SwipeRefreshLayout</code> .
03	Детайли на имот	Нова <code>EstateDetailActivity</code> . Закачаме я към клик на ред в списъка.
04	Добавяне на имот	Подменяме <code>SlideshowFragment</code> с <code>AddEstateFragment</code> . Добавяме <code>READ_MEDIA_IMAGES</code> permission, разширяваме <code>ApiClient</code> с <code>postMultipart</code> .
05	Карта	Подменяме <code>GalleryFragment</code> с <code>MapFragment</code> . Добавяме библиотека <code>osmdroid</code> и <code>ACCESS_NETWORK_STATE</code> permission.
06	Оферти	Нов екран <code>OffersFragment</code> като 4-ти елемент в <code>drawer-a</code> .
07	Чат	Нова <code>ChatActivity</code> . Закачаме я от <code>Offers</code> .
08	Logout	Последен щрих — <code>Logout</code> в <code>drawer-a</code> .

Важно: всеки урок е самостоятелен. Добавяме библиотеките, разрешенията, низовете и моделите **точно когато ни потрябват** — не ги подготвяме предварително.

Конвенции на API-то

Запомнете:

1. Всички пътища започват с `/api/...` (например `/api/estates`).
2. JSON полета са в **camelCase** — `firstName`, `categoryId`, `isSeller`.
3. Защитените пътища изискват `Authorization: Bearer<token>` — `ApiClient` го добавя автоматично, ако подадете `token` параметър.

Готови сме

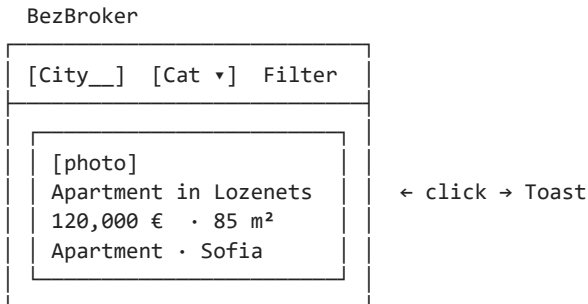
Започваме с Урок 02 — първият истински екран.

Урок 02 — Списък с имоти

Цел

Подменяме placeholder-а HomeFragment с реален екран — списък от имоти, дошли от API-то, с филтри по град и категория и pull-to-refresh.

Кликът върху ред засега ще показва само Toast — в **Урок 03** ще го свържем с екрана за детайли.



Какво ще променим в този урок

Действие	Файл
Добавяме	библиотеки: RecyclerView, Glide, SwipeRefreshLayout
Добавяме	низове за филтър
Добавяме	модели Estate и Category
Добавяме	EstateAdapter + item_estate.xml
Разширяваме	ApiClient — нов вариант на get с query параметри
Подменяме	HomeFragment → EstatesFragment, fragment_home.xml → fragment_estates.xml
Обновяваме	drawer меню, nav graph, MainActivity, strings.xml
Изтриваме	HomeFragment.java, HomeViewModel.java, fragment_home.xml

Стъпка 1 — Добавяне на библиотеките

RecyclerView е стандартният Android начин за дълги списъци. Glide сваля и кешира снимки от URL. SwipeRefreshLayout дава pull-to-refresh жест.

1a. gradle/libs.versions.toml

Намерете блока [versions] и **добавете накрая**:

```
recyclerview = "1.3.2"
swiperefresh = "1.1.0"
glide = "4.16.0"
```

В блока `[libraries]` **добавете накрая**:

```
recyclerview = { group = "androidx.recyclerview", name = "recyclerview", version.ref = "recyclerview" }
swiperefresh = { group = "androidx.swiperefreshlayout", name = "swiperefreshlayout", version.ref = "swiperefresh" }
glide = { group = "com.github.bumptech.glide", name = "glide", version.ref = "glide" }
```

1b. app/build.gradle

Намерете блока `dependencies { ... }`. След последния `implementation libs.navigation.ui` ред **добавете**:

```
implementation libs.recyclerview
implementation libs.swiperefresh
implementation libs.glide
```

Android Studio ще покаже жълта лента "Sync Now" — натиснете я. (Или `File` → `Sync Project with Gradle Files`.)

Стъпка 2 — Низовете за този екран

Файл: `app/src/main/res/values/strings.xml`

Намерете низовете `menu_home`, `menu_gallery`, `menu_slideshow` (вече има в шаблона). Преименуваме `menu_home`:

```
<!-- Преди: -->
<string name="menu_home">Home</string>
<!-- След: -->
<string name="menu_estates">Estates</string>
```

`menu_gallery` и `menu_slideshow` оставяме както са — ще ги преименуваме в Урок 04 и 05.

В края, преди `</resources>`, **добавете**:

```
<!-- Estates list -->
<string name="filter_city_hint">City</string>
<string name="filter_apply">Filter</string>
<string name="filter_any_category">Any category</string>
```

Стъпка 3 — Моделните класове

POJO класове, които прави обектите от JSON-а на сървъра.

Първо създайте нов пакет: десен бутон върху `com.example.bezbroker` в Project панела → `New` → `Package` → име `model`.

3a. model/Estate.java

```
package com.example.bezbroker.model;

import org.json.JSONArray;
import org.json.JSONException;
import org.json.JSONObject;

import java.util.ArrayList;
import java.util.List;

public class Estate {
    public int id;
    public int categoryId;
    public String categoryName;
    public String title;
    public String description;
    public String city;
    public double price;
    public double area;
    public String address;
    public Double lat;
    public Double lont;
    public int userId;
    public String createdAt;
    public List<String> photos = new ArrayList<>();

    // Owner info (came от GET /api/estates/{id})
    public String ownerEmail;
    public String ownerFirstName;
    public String ownerLastName;

    public static Estate fromJson(JSONObject o) throws JSONException {
        Estate e = new Estate();
        e.id = o.optInt("id");
        e.categoryId = o.optInt("categoryId");
        e.categoryName = o.optString("categoryName", "");
        e.title = o.optString("title", "");
        e.description = o.optString("description", "");
        e.city = o.optString("city", "");
        e.price = o.optDouble("price", 0);
        e.area = o.optDouble("area", 0);
        e.address = o.optString("address", "");
        e.lat = o.isNull("lat") ? null : o.optDouble("lat");
        e.lont = o.isNull("lont") ? null : o.optDouble("lont");
        e.userId = o.optInt("userId");
        e.createdAt = o.optString("createdAt", "");

        e.ownerEmail = o.optString("ownerEmail", "");
        e.ownerFirstName = o.optString("ownerFirstName", "");
        e.ownerLastName = o.optString("ownerLastName", "");

        JSONArray photos = o.optJSONArray("photos");
        if (photos != null) {
            for (int i = 0; i < photos.length(); i++) {
                e.photos.add(photos.optString(i));
            }
        }
        return e;
    }

    public static List<Estate> listFromJson(JSONArray arr) throws JSONException {
        List<Estate> out = new ArrayList<>();
        if (arr == null) return out;
        for (int i = 0; i < arr.length(); i++) out.add(fromJson(arr.getJSONObject(i)));
        return out;
    }
}
```

```

    }

    public String firstPhotoOrNull() {
        return photos.isEmpty() ? null : photos.get(0);
    }
}

```

Обяснение:

- `lat/lont` са `Double` (с главно D) и **могат да са** `null` — ако имотът няма координати.
- `optString("createdAt", "")` — никога не хвърля грешка; връща втория аргумент ако ключът липсва.
- `firstPhotoOrNull()` ще ползваме за главната снимка в списъка.

3b. model/Category.java

```

package com.example.bezbroker.model;

import org.json.JSONArray;
import org.json.JSONException;
import org.json.JSONObject;

import java.util.ArrayList;
import java.util.List;

public class Category {
    public int id;
    public String name;

    public Category(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public static List<Category> listFromJson(JSONArray arr) throws JSONException {
        List<Category> out = new ArrayList<>();
        if (arr == null) return out;
        for (int i = 0; i < arr.length(); i++) {
            JSONObject o = arr.getJSONObject(i);
            out.add(new Category(o.optInt("id"), o.optString("name")));
        }
        return out;
    }

    @Override public String toString() { return name; }
}

```

`toString()` връща `name`, защото ще ползваме `Category` в `Spinner` — Android по подразбиране показва `toString()` за всеки елемент.

Стъпка 4 — Разширяване на `ApiClient` с `GET + query`

Текущо `ApiClient.get` приема само `(path, token, callback)`. За филтрите ни трябва вариант с `query` параметри (`?city=Sofia&categoryId=1`).

Файл: `app/src/main/java/com/example/bezbroker/ApiClient.java`

Добавете тези `import`-и в началото (ако вече ги има, пропуснете):

```

import java.net.URLEncoder;
import java.util.Map;

```

Под съществуващия метод:

```
public static void get(String path, String token, Callback cb){
    request("GET", path, null, token, cb);
}
```

добавете:

```
/** GET with query parameters appended as ?k=v&k2=v2. */
public static void get(String path, Map<String, String> query, String token, Callback cb){
    request("GET", path + buildQuery(query), null, token, cb);
}
```

И някъде в края на класа (преди затварящата скоба }), добавете малката помощна функция:

```
private static String buildQuery(Map<String, String> q) {
    if (q == null || q.isEmpty()) return "";
    StringBuilder sb = new StringBuilder("?");
    boolean first = true;
    for (Map.Entry<String, String> e : q.entrySet()) {
        if (e.getValue() == null) continue;
        if (!first) sb.append("&");
        try {
            sb.append(URLEncoder.encode(e.getKey(), "UTF-8"))
                .append("=")
                .append(URLEncoder.encode(e.getValue(), "UTF-8"));
        } catch (Exception ignored) { }
        first = false;
    }
    return sb.toString();
}
```

Защо два метода get с едно и също име: Java поддържа **method overloading** — два метода с едно име, ако имат различни параметри. Java сам избира правилния според това какво подаваме.

Стъпка 5 — Изглед на един ред (item_estate.xml)

В Project панела: десен бутон върху res/layout → New → Layout Resource File → име item_estate, root element com.google.android.material.card.MaterialCardView.

После заместете цялото съдържание с:

```
<?xml version="1.0" encoding="utf-8"?>
<com.google.android.material.card.MaterialCardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="8dp"
    app:cardCornerRadius="8dp"
    app:cardElevation="2dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="8dp">

        <ImageView
            android:id="@+id/itemEstateImage"
            android:layout_width="match_parent"
            android:layout_height="160dp"
            android:background="#EEEEEE"
            android:scaleType="centerCrop" />

        <TextView
            android:id="@+id/itemEstateTitle"
```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:textSize="16sp"
        android:textStyle="bold" />

<TextView
    android:id="@+id/itemEstatePrice"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textColor="#2941CE"
    android:textSize="14sp" />

<TextView
    android:id="@+id/itemEstateMeta"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textColor="#666666"
    android:textSize="12sp" />
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

```

`MaterialCardView` — карта със заоблени ъгли и лека сянка. Сивият фон #EEEEEE на `ImageView`-то е placeholder, докато `Glide` зареди снимката.

Стъпка 6 — EstateAdapter

Първо създайте пакет `adapter`: десен бутон върху `com.example.bezbroker` → New → Package → `adapter`.

Файл: `adapter/EstateAdapter.java`

```

package com.example.bezbroker.adapter;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ImageView;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.bumptech.glide.Glide;
import com.example.bezbroker.R;
import com.example.bezbroker.model.Estate;

import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

public class EstateAdapter extends RecyclerView.Adapter<EstateAdapter.VH> {

    public interface OnClick { void onEstateClick(Estate estate); }

    private final List<Estate> items = new ArrayList<>();
    private final OnClick onClick;

    public EstateAdapter(OnClick onClick) {
        this.onClick = onClick;
    }

    public void setItems(List<Estate> list) {
        items.clear();
        items.addAll(list);
    }

```

```

        notifyDataSetChanged();
    }

    @NonNull @Override
    public VH onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
        View v = LayoutInflater.from(parent.getContext()).inflate(R.layout.item_estate, parent, false);
        return new VH(v);
    }

    @Override
    public void onBindViewHolder(@NonNull VH h, int position) {
        Estate e = items.get(position);
        h.title.setText(e.title);
        h.price.setText(String.format(Locale.getDefault(),
            "%.0f € · %.0f m²", e.price, e.area));
        h.meta.setText(String.format(Locale.getDefault(),
            "%s · %s", e.categoryName, e.city));

        String photo = e.firstPhotoOrNull();
        if (photo != null) {
            Glide.with(h.image).load(photo).into(h.image);
        } else {
            h.image.setImageDrawable(null);
        }
        h.itemView.setOnClickListener(v -> onClick.onEstateClick(e));
    }

    @Override public int getItemCount() { return items.size(); }

    static class VH extends RecyclerView.ViewHolder {
        ImageView image;
        TextView title, price, meta;
        VH(@NonNull View v) {
            super(v);
            image = v.findViewById(R.id.itemEstateImage);
            title = v.findViewById(R.id.itemEstateTitle);
            price = v.findViewById(R.id.itemEstatePrice);
            meta = v.findViewById(R.id.itemEstateMeta);
        }
    }
}

```

Концепциите за студентите:

- RecyclerView рециклира view-овете — само 5-10 копия от `item_estate.xml` се правят и при скрол се пренареждат с нови данни.
- ViewHolder (VH) е малък пакет, в който държим `findViewById` референциите, за да не ги викаме при всеки скрол.
- `onCreateViewHolder` се вика само когато трябва **нов** view (например при инициализиране).
- `onBindViewHolder` се вика **често** — всеки път, когато view трябва да покаже нови данни.
- `Glide.with(imageView).load(url).into(imageView)` — една магическа линия: сваля снимката, кешира я, слага я в ImageView-то.
- `onClick` **интерфейс**: адаптерът не знае какво трябва да се случи при клик; fragment-ът ще му каже.

Стъпка 7 — Изтриване на HomeFragment и създаване на EstatesFragment

7а. Изтриване

В Project панела:

- Десен бутон върху пакета `com.example.bezbroker.ui.home` → Delete. (Това трие `HomeFragment.java` + `HomeViewModel.java`.)
- Десен бутон върху `res/layout/fragment_home.xml` → Delete.

Android Studio може да попита "Find usages" — натиснете Delete Anyway (ще оправим референциите след малко).

7b. Създаване на нов пакет

Десен бутон върху `com.example.bezbroker.ui` → New → Package → `estates`.

7c. Нов layout `fragment_estates.xml`

В Project панела: десен бутон върху `res/layout` → New → Layout Resource File → име `fragment_estates`, root element `LinearLayout`.

После заменете съдържанието с:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".ui.estates.EstatesFragment">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:padding="8dp">

        <EditText
            android:id="@+id/filterCityET"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="2"
            android:hint="@string/filter_city_hint"
            android:imeOptions="actionSearch"
            android:inputType="text" />

        <Spinner
            android:id="@+id/filterCategorySpinner"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginStart="8dp"
            android:layout_weight="1" />

        <Button
            android:id="@+id/filterApplyB"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/filter_apply" />
    </LinearLayout>

    <androidx.swiperefreshlayout.widget.SwipeRefreshLayout
        android:id="@+id/swipeRefresh"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1">

        <androidx.recyclerview.widget.RecyclerView
            android:id="@+id/estatesRV"
            android:layout_width="match_parent"
            android:layout_height="match_parent">
```

```

        android:clipToPadding="false"
        android:padding="8dp" />
    </androidx.swiperefreshlayout.widget.SwipeRefreshLayout>
</LinearLayout>

```

Два реда: горен с филтър (City weight=2, Spinner weight=1, Button), долен с SwipeRefreshLayout обвиващ RecyclerView.

7d. Нов клас EstatesFragment.java

В новия пакет ui.estates: десен бутон → New → Java Class → EstatesFragment.

```

package com.example.bezbroker.ui.estates;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.AdapterView;
import android.widget.ArrayAdapter;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Spinner;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;
import androidx.swiperefreshlayout.widget.SwipeRefreshLayout;

import com.example.bezbroker.ApiClient;
import com.example.bezbroker.R;
import com.example.bezbroker.SessionManager;
import com.example.bezbroker.adapter.EstateAdapter;
import com.example.bezbroker.model.Category;
import com.example.bezbroker.model.Estate;

import org.json.JSONObject;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class EstatesFragment extends Fragment {

    private EstateAdapter adapter;
    private SwipeRefreshLayout swipe;
    private EditText cityET;
    private Spinner categorySpinner;
    private List<Category> categories = new ArrayList<>();

    @Override
    public View onCreateView(@NonNull LayoutInflater inflater,
                             @Nullable ViewGroup container,
                             @Nullable Bundle savedInstanceState) {
        View root = inflater.inflate(R.layout.fragment_estates, container, false);

        cityET = root.findViewById(R.id.filterCityET);
        categorySpinner = root.findViewById(R.id.filterCategorySpinner);
        Button applyB = root.findViewById(R.id.filterApplyB);
        swipe = root.findViewById(R.id.swipeRefresh);
        RecyclerView rv = root.findViewById(R.id.estatesRV);

        rv.setLayoutManager(new LinearLayoutManager(requireContext()));
    }

```

```

        adapter = new EstateAdapter(estate -> {
            // Placeholder – в Урок 03 ще отворим EstateDetailActivity
            Toast.makeText(requireContext(),
                "Click on estate id=" + estate.id, Toast.LENGTH_SHORT).show();
        });
        rv.setAdapter(adapter);

        swipe.setOnRefreshListener(this::loadEstates);
        applyB.setOnClickListener(v -> loadEstates());

        loadCategories();
        return root;
    }

    @Override
    public void onResume() {
        super.onResume();
        loadEstates();
    }

    private void loadCategories() {
        ApiClient.get("/api/categories", null, new ApiClient.Callback() {
            @Override public void onSuccess(JSONObject body) {
                try {
                    categories = Category.listFromJson(body.optJSONArray("categories"));
                    List<String> names = new ArrayList<>();
                    names.add(getString(R.string.filter_any_category));
                    for (Category c : categories) names.add(c.name);
                    ArrayAdapter<String> ad = new ArrayAdapter<>(
                        requireContext(), android.R.layout.simple_spinner_dropdown_item, names);
                    categorySpinner.setAdapter(ad);
                } catch (Exception ignored) { }
            }
            @Override public void onError(int httpCode, String message) { /* non-fatal */ }
        });
    }

    private void loadEstates() {
        swipe.setRefreshing(true);
        String token = new SessionManager(requireContext()).getToken();
        Map<String, String> q = new HashMap<>();
        String city = cityET.getText().toString().trim();
        if (!city.isEmpty()) q.put("city", city);
        int catPos = categorySpinner.getSelectedItemPosition();
        if (catPos > 0 && catPos - 1 < categories.size()) {
            q.put("categoryId", String.valueOf(categories.get(catPos - 1).id));
        }
        ApiClient.get("/api/estates", q, token, new ApiClient.Callback() {
            @Override public void onSuccess(JSONObject body) {
                swipe.setRefreshing(false);
                try {
                    adapter.setItems(Estate.listFromJson(body.optJSONArray("estates")));
                } catch (Exception e) {
                    Toast.makeText(requireContext(), e.getMessage(), Toast.LENGTH_SHORT).show();
                }
            }
            @Override public void onError(int httpCode, String message) {
                swipe.setRefreshing(false);
                Toast.makeText(requireContext(), message, Toast.LENGTH_LONG).show();
            }
        });
    }
}

```

Обяснение:

- `onCreateView` — вика се веднъж когато се инстанцира `fragment`-ът. Тук `inflate`-ваме `layout`-а и регистрираме `listener`-и.
- `onResume` — вика се всеки път когато екранът става видим. Презареждаме списъка тук, за да хванем новопоявили се имоти.
- `requireContext()` — във `Fragment context`-ът може да е `null` между `onAttach` и `onDetach`. `requireContext()` хвърля грешка вместо тихо `null`.
- **Click handler-ът засега показва само Toast** — в Урок 03 ще го сменим да отваря `EstateDetailActivity`.

Стъпка 8 — Drawer меню, nav graph, MainActivity

`HomeFragment` беше първият елемент в `drawer`-а. Сега го превръщаме в "Estates".

8a. `res/menu/activity_main_drawer.xml`

Намерете първия `<item>`:

```
<item
    android:id="@+id/nav_home"
    android:icon="@drawable/ic_menu_camera"
    android:title="@string/menu_home" />
```

Заменете с:

```
<item
    android:id="@+id/nav_estates"
    android:icon="@drawable/ic_menu_camera"
    android:title="@string/menu_estates" />
```

Двата други (`nav_gallery`, `nav_slideshow`) не пипаме.

8b. `res/navigation/mobile_navigation.xml`

Сменете `app:startDestination`:

```
app:startDestination="@+id/nav_estates"
```

Сменете първия `<fragment>`:

```
<fragment
    android:id="@+id/nav_estates"
    android:name="com.example.bezbroker.ui.estates.EstatesFragment"
    android:label="@string/menu_estates"
    tools:layout="@layout/fragment_estates" />
```

Останалите два не пипаме.

8c. `MainActivity.java`

В `onCreate` намерете:

```
mAppBarConfiguration = new AppBarConfiguration.Builder(
    R.id.nav_home, R.id.nav_gallery, R.id.nav_slideshow)
    .setOpenableLayout(drawer)
    .build();
```

Сменете `R.id.nav_home` с `R.id.nav_estates`:

```
mAppBarConfiguration = new AppBarConfiguration.Builder(
    R.id.nav_estates, R.id.nav_gallery, R.id.nav_slideshow)
    .setOpenableLayout(drawer)
    .build();
```

Тест

Build → Make Project. Трябва да компилира.

Run. След login виждате:

- Drawer-а — три елемента: **Estates** · Gallery · Slideshow.
- Estates е startDestination → отваря се пръв.
- Списък с имоти (или празен, ако още няма такива в API-то).
- Дръпни надолу → reload.
- Напиши "Sofia" в полето → Filter → списъкът се филтрира.
- Клик на ред → Toast "Click on estate id=X".

Gallery и Slideshow в drawer-а още работят като placeholder-и — нормално, ще ги подменим в Уроци 04 и 05.

Често срещани грешки

Симптом	Причина
Compile грешка Cannot resolve symbol 'menu_home'	Drawer XML или nav graph все още използва menu_home или nav_home.
ClassNotFoundException: EstatesFragment	Пакетът трябва да е точно com.example.bezbroker.ui.estates.
Снимките не се зареждат	Glide изисква INTERNET permission (вече я има в манифеста от шаблона) и валидни URL-и.

Следва

В **Урок 03** правим EstateDetailActivity и заменяме Toast-а с реално отваряне на детайла.

Урок 03 — Детайли на имот

Цел

Когато потребителят натисне ред от списъка, отваряме нов екран с:

- карусел от снимки;
- заглавие, цена, площ;
- категория · град · адрес;
- описание;
- инфо за собственика;
- бутон "Make offer" (само ако не сме собственикът).

Натискане на "Make offer" → диалог за сума → POST /api/offers.

Защо Activity, а не Fragment?

Това е "дълбок" екран — отваря се при tap, не от drawer-a. Очакваме стрелка ◀ горе вляво (за връщане назад), не drawer ◻. Затова е обикновен AppCompatActivity.

Какво ще променим

Действие	Файл
Добавяме	низове за detail и offer
Добавяме	EstateDetailActivity в манифеста
Създаваме	activity_estate_detail.xml
Създаваме	EstateDetailActivity.java
Обновяваме	click handler-a в EstatesFragment (Toast → startActivity)

Glide и моделът Estate ги добавихме в Урок 02, така че няма да ги повтаряме.

Стъпка 1 — Низове за този екран

Файл: res/values/strings.xml

Преди </resources> **добавете**:

```
<!-- Estate detail -->
<string name="make_offer">Make offer</string>
<string name="offer_sum_hint">Your offer (€)</string>
<string name="send_offer">Send</string>
```

cancel вече го има от шаблона.

Стъпка 2 — Регистриране в манифеста

Файл: app/src/main/AndroidManifest.xml

В <application> блока, до другите <activity> декларации, **добавете**:

```
<activity
    android:name=".EstateDetailActivity"
    android:exported="false" />
```

android:exported="false" — само нашето приложение може да я отвори, не външни приложения.

Стъпка 3 — Layout activity_estate_detail.xml

res/layout → десен бутон → New → Layout Resource File → име activity_estate_detail, root element ScrollView.

Заместете съдържанието с:

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <HorizontalScrollView
            android:layout_width="match_parent"
            android:layout_height="220dp">

            <LinearLayout
                android:id="@+id/detailPhotosContainer"
                android:layout_width="wrap_content"
                android:layout_height="match_parent"
                android:orientation="horizontal" />
        </HorizontalScrollView>

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="16dp">

            <TextView
                android:id="@+id/detailTitleTV"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:textSize="20sp"
                android:textStyle="bold" />

            <TextView
                android:id="@+id/detailPriceTV"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="4dp"
                android:textColor="#2941CE"
                android:textSize="16sp" />

            <TextView
                android:id="@+id/detailMetaTV"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:textColor="#666666" />

            <TextView
```

```

        android:id="@+id/detailDescriptionTV"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp" />

        <TextView
            android:id="@+id/detailOwnerTV"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="12dp"
            android:textColor="#666666"
            android:textSize="12sp" />

        <Button
            android:id="@+id/detailMakeOfferB"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
            android:text="@string/make_offer" />
    </LinearLayout>
</LinearLayout>
</ScrollView>

```

Структура:

- Външно: `ScrollView` (целият екран скролва вертикално).
- Вътрешно вертикален `LinearLayout`:
- Горе: `HorizontalScrollView` 220dp висок, с `LinearLayout` хоризонтален → каруселът от снимки.
- Долу: блок с `padding 16dp` — заглавие, цена, мета, описание, собственик, бутон.

Стъпка 4 — EstateDetailActivity.java

В пакета `com.example.bezbroker` (където са `LoginActivity`, `MainActivity` и т.н.): десен бутон → New → Java Class → `EstateDetailActivity`.

```

package com.example.bezbroker;

import android.app.AlertDialog;
import android.os.Bundle;
import android.text.InputType;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ImageView;
import android.widget.LinearLayout;
import android.widget.TextView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

import com.bumptech.glide.Glide;
import com.example.bezbroker.model.Estate;

import org.json.JSONObject;

import java.util.Locale;

public class EstateDetailActivity extends AppCompatActivity {

    private TextView titleTV, priceTV, metaTV, descTV, ownerTV;
    private LinearLayout photosContainer;
    private Estate currentEstate;

```

```

private int estateId;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_estate_detail);

    estateId = getIntent().getIntExtra("estate_id", 0);
    if (estateId == 0) { finish(); return; }

    titleTV = findViewById(R.id.detailTitleTV);
    priceTV = findViewById(R.id.detailPriceTV);
    metaTV = findViewById(R.id.detailMetaTV);
    descTV = findViewById(R.id.detailDescriptionTV);
    ownerTV = findViewById(R.id.detailOwnerTV);
    photosContainer = findViewById(R.id.detailPhotosContainer);

    Button makeOfferB = findViewById(R.id.detailMakeOfferB);
    makeOfferB.setOnClickListener(v -> promptForOffer());

    load();
}

private void load() {
    String token = new SessionManager(this).getToken();
    ApiClient.get("/api/estates/" + estateId, token, new ApiClient.Callback() {
        @Override public void onSuccess(JSONObject body) {
            try {
                JSONObject o = body.optJSONObject("estate");
                if (o == null) { finish(); return; }
                currentEstate = Estate.fromJson(o);
                bind();
            } catch (Exception e) {
                Toast.makeText(EstateDetailActivity.this, e.getMessage(), Toast.LENGTH_LONG).show();
            }
        }
        @Override public void onError(int httpCode, String message) {
            Toast.makeText(EstateDetailActivity.this, message, Toast.LENGTH_LONG).show();
            finish();
        }
    });
}

private void bind() {
    titleTV.setText(currentEstate.title);
    priceTV.setText(String.format(Locale.getDefault(),
        "%,.0f € · %.0f m²", currentEstate.price, currentEstate.area));
    metaTV.setText(String.format(Locale.getDefault(),
        "%s · %s%s",
        currentEstate.categoryName,
        currentEstate.city,
        currentEstate.address.isEmpty() ? "" : " · " + currentEstate.address));
    descTV.setText(currentEstate.description);

    String ownerName = (currentEstate.ownerFirstName + " " + currentEstate.ownerLastName).trim();
    if (ownerName.isEmpty()) ownerName = currentEstate.ownerEmail;
    ownerTV.setText("Listed by: " + ownerName);

    photosContainer.removeAllViews();
    for (String url : currentEstate.photos) {
        ImageView iv = new ImageView(this);
        iv.setLayoutParams(new ViewGroup.LayoutParams(
            (int) (220 * getResources().getDisplayMetrics().density),
            ViewGroup.LayoutParams.MATCH_PARENT));
        iv.setScaleType(ImageView.ScaleType.CENTER_CROP);
        iv.setPadding(0, 0, 8, 0);
    }
}

```

```

        Glide.with(this).load(url).into(iv);
        photosContainer.addView(iv);
    }

    // Скриваме "Make offer" ако сме собственикът.
    Button b = findViewById(R.id.detailMakeOfferB);
    int myId = currentUserId();
    b.setVisibility(myId == currentEstate.userId ? View.GONE : View.VISIBLE);
}

private void promptForOffer() {
    EditText input = new EditText(this);
    input.setInputType(InputType.TYPE_CLASS_NUMBER | InputType.TYPE_NUMBER_FLAG_DECIMAL);
    input.setHint(R.string.offer_sum_hint);

    new AlertDialog.Builder(this)
        .setTitle(R.string.make_offer)
        .setView(input)
        .setPositiveButton(R.string.send_offer, (d, w) -> {
            String txt = input.getText().toString().trim();
            if (!txt.isEmpty()) sendOffer(Double.parseDouble(txt));
        })
        .setNegativeButton(R.string.cancel, null)
        .show();
}

private void sendOffer(double sum) {
    JSONObject body = new JSONObject();
    try {
        body.put("estateId", estateId);
        body.put("sum", sum);
    } catch (Exception ignored) { }

    String token = new SessionManager(this).getToken();
    ApiClient.post("/api/offers", body, token, new ApiClient.Callback() {
        @Override public void onSuccess(JSONObject body) {
            Toast.makeText(EstateDetailActivity.this, "Offer sent", Toast.LENGTH_SHORT).show();
            finish();
        }
        @Override public void onError(int httpCode, String message) {
            Toast.makeText(EstateDetailActivity.this, message, Toast.LENGTH_LONG).show();
        }
    });
}

private int currentUserId() {
    try {
        String json = new SessionManager(this).getUserJson();
        if (json == null) return 0;
        return new JSONObject(json).optInt("id", 0);
    } catch (Exception e) { return 0; }
}
}

```

Какво се случва, метод по метод:

Метод	Какво прави
onCreate	Чете estate_id от Intent-а, намира view-овете, регистрира click handler за бутона, стартира load().
load()	GET /api/estates/{id} → попълва currentEstate → вика bind().

Метод	Какво прави
<code>bind()</code>	Слага текстовете, динамично добавя <code>ImageView</code> -та за всяка снимка, скрива бутон "Make offer" ако сме собственикът.
<code>promptForOffer()</code>	Показва <code>AlertDialog</code> с поле за сума. При ОК → <code>sendOffer</code> .
<code>sendOffer(sum)</code>	<code>POST /api/offers</code> с тяло <code>{estateId, sum}</code> . При успех → затваря екрана.
<code>currentUserId()</code>	Парсва запазения <code>user JSON</code> от <code>SessionManager</code> .

Важни детайли:

- `getIntent().getIntExtra("estate_id", 0)` — взимаме `id`-то, което `EstatesFragment` ни подава. Втория аргумент е `default` ако ключът липсва.
- `(int) (220 * getResources().getDisplayMetrics().density)` — превръща **220 dp** в реални **pixel**-и. Без този коефициент картинките ще са различно големи на различни устройства.
- `View.GONE` vs `View.INVISIBLE`:
- `GONE` — `view`-ът изчезва напълно (други `view`-ове запълват пространството).
- `INVISIBLE` — `view`-ът е невидим, но запазва пространството.

Стъпка 5 — Подмяна на `Toast`-а в `EstatesFragment`

Файл: `ui/estates/EstatesFragment.java`

В `onCreateView` намерете `click handler`-а от Урок 02:

```
adapter = new EstateAdapter(estate -> {
    // Placeholder – в Урок 03 ще отворим EstateDetailActivity
    Toast.makeText(requireContext(),
        "Click on estate id=" + estate.id, Toast.LENGTH_SHORT).show();
});
```

Заменете с:

```
adapter = new EstateAdapter(estate -> {
    Intent i = new Intent(requireContext(), EstateDetailActivity.class);
    i.putExtra("estate_id", estate.id);
    startActivity(i);
});
```

Добавете тези `import`-и в горната част на файла (ако ги няма):

```
import android.content.Intent;
import com.example.bezbroker.EstateDetailActivity;
```

Можете да премахнете `import android.widget.Toast`; ако `Toast` вече не се ползва никъде във файла.

Тест

1. Run.
2. От списъка натиснете на имот → отваря се новият екран.
3. Виждате снимки (ако са качени), цена, описание, собственик.
4. Ако имотът **не** е ваш → бутон "Make offer". Натиснете → попълнете сума → Send → Toast "Offer sent".

Засега не виждаме направената оферта — OffersFragment ще го направим в Урок 06.

Често срещани грешки

Симптом	Причина
App crash при клик	Не сте регистрирали EstateDetailActivity в манифеста (Стъпка 2).
HTTP 404	estateId == 0 или подаден грешен id.
Каруселът не скролва хоризонтално	HorizontalScrollView обвива LinearLayout с orientation="horizontal" — проверете orientation-a.
Make offer не изчезва за моя имот	currentUserId() връща 0. Проверете SessionManager.getUserJson().

Следва

В **Урок 04** правим AddEstateFragment — добавяне на имот със снимки.

Урок 04 — Добавяне на имот

Цел

Подменяме placeholder-а SlideshowFragment с реален екран — форма за добавяне на имот + photo picker + два upload-а (първо имота, после снимките като multipart).

Какво ще променим

Действие	Файл
Добавяме	READ_MEDIA_IMAGES permission
Разширяваме	ApiClient с postMultipart + клас FilePart
Добавяме	низове за този екран
Подменяме	SlideshowFragment → AddEstateFragment
Обновяваме	drawer меню, nav graph, MainActivity
Обновяваме	FAB бутона да отваря Add estate (вместо placeholder Snackbar-а)
Изтриваме	SlideshowFragment.java, SlideshowViewModel.java, fragment_slideshow.xml

Estate и Category модели ги имаме от Урок 02.

Стъпка 1 — Permission за достъп до галерията

Файл: AndroidManifest.xml

Намерете реда:

```
<uses-permission android:name="android.permission.INTERNET" />
```

Под него добавете:

```
<uses-permission android:name="android.permission.READ_MEDIA_IMAGES" />
```

Защо: Android 13+ изисква специално разрешение за четене на снимки от галерията.

Стъпка 2 — postMultipart метод в ApiClient

Файл: ApiClient.java

Снимките се качват в формат multipart/form-data — различен от JSON. Не ползваме библиотека, така че го пишем ръчно.

Добавете тези import-и в горната част на файла:

```
import java.io.ByteArrayOutputStream;
import java.io.DataOutputStream;
import java.nio.charset.StandardCharsets;
import java.util.List;
```

Под съществуващия метод `get(String path, Map<String, String> query, ...)` (от Урок 02) **добавете:**

```
/** Multipart upload – used for posting photos. */
public static void postMultipart(String path,
                                Map<String, String> fields,
                                List<FilePart> files,
                                String token,
                                Callback cb) {
    EXEC.execute(() -> {
        HttpURLConnection con = null;
        try {
            URL url = new URL(BASE_URL + path);
            con = (HttpURLConnection) url.openConnection();
            con.setRequestMethod("POST");
            con.setConnectTimeout(60000);
            con.setReadTimeout(60000);
            con.setDoOutput(true);
            con.setUseCaches(false);

            if (token != null) {
                con.setRequestProperty("Authorization", "Bearer" + token);
            }
            con.setRequestProperty("Accept", "application/json");

            String boundary = "----BezBroker" + System.currentTimeMillis();
            con.setRequestProperty("Content-Type", "multipart/form-data; boundary=" + boundary);

            try (DataOutputStream out = new DataOutputStream(con.getOutputStream())) {
                if (fields != null) {
                    for (Map.Entry<String, String> e : fields.entrySet()) {
                        out.writeBytes("--" + boundary + "\r\n");
                        out.writeBytes("Content-Disposition: form-data; name=\"" + e.getKey() + "\"\r\n\r\n");
                        out.write(e.getValue().getBytes(StandardCharsets.UTF_8));
                        out.writeBytes("\r\n");
                    }
                }
                if (files != null) {
                    for (FilePart f : files) {
                        out.writeBytes("--" + boundary + "\r\n");
                        out.writeBytes("Content-Disposition: form-data; name=\"" + f.fieldName +
                                      "\"; filename=\"" + f.fileName + "\"\r\n");
                        out.writeBytes("Content-Type: " + f.mime + "\r\n\r\n");
                        out.write(f.bytes);
                        out.writeBytes("\r\n");
                    }
                }
                out.writeBytes("--" + boundary + "--\r\n");
            }

            int code = con.getResponseCode();
            InputStream stream = (code >= 200 && code < 300) ? con.getInputStream() : con.getErrorStream();
            String text = readAll(stream);
            JSONObject json;
            try { json = text.isEmpty() ? new JSONObject() : new JSONObject(text); }
            catch (JSONException pe) { json = new JSONObject(); }
            final int finalCode = code;
            final JSONObject finalJson = json;
            if (code >= 200 && code < 300) {
                MAIN.post(() -> cb.onSuccess(finalJson));
            } else {
                MAIN.post(() -> cb.onError(finalCode, finalJson.optString("message", "Upload failed")));
            }
        } catch (IOException e) {
            MAIN.post(() -> cb.onError(0, e.getMessage() == null ? "Network error" : e.getMessage()));
        } finally {

```

```

        if (con != null) con.disconnect();
    }
});
}

```

И някъде в края на класа (преди затварящата `}`) **добавете** малкия вложен клас:

```

/** One file in a multipart request. */
public static class FilePart {
    public final String fieldName;    // server field, например "photos"
    public final String fileName;    // показва се на сървъра, например "photo.jpg"
    public final String mime;        // image/jpeg | image/png | image/webp
    public final byte[] bytes;
    public FilePart(String fieldName, String fileName, String mime, byte[] bytes) {
        this.fieldName = fieldName; this.fileName = fileName; this.mime = mime; this.bytes = bytes;
    }
}

```

Какво прави този метод:

1. Старта работа на background thread (EXEC.execute).
2. Лепи URL-а, слага token-а.
3. Генерира уникален boundary низ — разделител между частите.
4. Пише тялото с DataOutputStream (полета + файлове + затваряща линия).
5. Чете отговора и го връща през callback-а на UI thread-а.

Стъпка 3 — Низове за този екран

Файл: res/values/strings.xml

Намерете menu_slideshow:

```
<string name="menu_slideshow">Slideshow</string>
```

Преименуваме го:

```
<string name="menu_add_estate">Add estate</string>
```

Преди `</resources>` **добавете**:

```

<!-- Add estate -->
<string name="add_estate_title_hint">Title</string>
<string name="add_estate_description_hint">Description</string>
<string name="add_estate_city_hint">City</string>
<string name="add_estate_address_hint">Address</string>
<string name="add_estate_price_hint">Price (€)</string>
<string name="add_estate_area_hint">Area (m²)</string>
<string name="add_estate_lat_hint">Latitude (optional)</string>
<string name="add_estate_lont_hint">Longitude (optional)</string>
<string name="add_estate_pick_photos">Pick photos</string>
<string name="add_estate_save">Save estate</string>
<string name="add_estate_saved">Estate saved</string>

```

Стъпка 4 — Изтриване на Slideshow placeholder-а

В Project панела:

- Десен бутон върху пакета `com.example.bezbroker.ui.slideshow` → Delete. (Трие SlideshowFragment.java + SlideshowViewModel.java.)
- Десен бутон върху `res/layout/fragment_slideshow.xml` → Delete.

Стъпка 5 — Нов пакет ui.add + нов layout

Десен бутон върху com.example.bezbroker.ui → New → Package → add.

После res/layout → New → Layout Resource File → име fragment_add_estate, root element ScrollView.

Заменете съдържанието с:

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".ui.add.AddEstateFragment">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <EditText
            android:id="@+id/addTitleET"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="@string/add_estate_title_hint"
            android:inputType="text" />

        <Spinner
            android:id="@+id/addCategorySpinner"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp" />

        <EditText
            android:id="@+id/addDescriptionET"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp"
            android:hint="@string/add_estate_description_hint"
            android:inputType="textMultiline"
            android:minLines="3" />

        <EditText
            android:id="@+id/addCityET"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp"
            android:hint="@string/add_estate_city_hint"
            android:inputType="text" />

        <EditText
            android:id="@+id/addAddressET"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp"
            android:hint="@string/add_estate_address_hint"
            android:inputType="text" />

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp"
            android:orientation="horizontal">
```

```

<EditText
    android:id="@+id/addPriceET"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_weight="1"
    android:hint="@string/add_estate_price_hint"
    android:inputType="numberDecimal" />

<EditText
    android:id="@+id/addAreaET"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_marginStart="8dp"
    android:layout_weight="1"
    android:hint="@string/add_estate_area_hint"
    android:inputType="numberDecimal" />
</LinearLayout>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:orientation="horizontal">

    <EditText
        android:id="@+id/addLatET"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:hint="@string/add_estate_lat_hint"
        android:inputType="numberSigned|numberDecimal" />

    <EditText
        android:id="@+id/addLontET"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginStart="8dp"
        android:layout_weight="1"
        android:hint="@string/add_estate_lont_hint"
        android:inputType="numberSigned|numberDecimal" />
</LinearLayout>

<Button
    android:id="@+id/addPickPhotosB"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:text="@string/add_estate_pick_photos" />

<TextView
    android:id="@+id/addPhotosCountTV"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="4dp"
    android:textColor="#666666" />

<Button
    android:id="@+id/addSaveB"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:text="@string/add_estate_save" />
</LinearLayout>
</ScrollView>

```

Стъпка 6 — AddEstateFragment.java

В пакета `ui.add`: десен бутон → New → Java Class → AddEstateFragment.

```
package com.example.bezbroker.ui.add;

import android.content.ContentResolver;
import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.webkit.MimeTypeMap;
import android.widget.AdapterView;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Spinner;
import android.widget.TextView;
import android.widget.Toast;

import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.navigation.Navigation;

import com.example.bezbroker.ApiClient;
import com.example.bezbroker.R;
import com.example.bezbroker.SessionManager;
import com.example.bezbroker.model.Category;

import org.json.JSONObject;

import java.io.ByteArrayOutputStream;
import java.io.InputStream;
import java.util.ArrayList;
import java.util.List;

public class AddEstateFragment extends Fragment {

    private EditText titleET, descET, cityET, addressET, priceET, areaET, latET, lontET;
    private Spinner categorySpinner;
    private TextView photosCountTV;
    private List<Category> categories = new ArrayList<>();
    private final List<Uri> pickedPhotos = new ArrayList<>();

    private final ActivityResultLauncher<Intent> pickPhotos =
        registerForActivityResult(new ActivityResultContracts.StartActivityForResult(), result -> {
            if (result.getResultCode() != requireActivity().RESULT_OK || result.getData() == null) return;
            Intent data = result.getData();
            pickedPhotos.clear();
            if (data.getClipData() != null) {
                for (int i = 0; i < data.getClipData().getItemCount(); i++) {
                    pickedPhotos.add(data.getClipData().getItemAt(i).getUri());
                }
            } else if (data.getData() != null) {
                pickedPhotos.add(data.getData());
            }
            photosCountTV.setText(getString(R.string.add_estate_pick_photos)
                + ": " + pickedPhotos.size());
        });

    @Override
    public View onCreateView(@NonNull LayoutInflater inflater,
```

```

        @Nullable ViewGroup container,
        @Nullable Bundle savedInstanceState) {
View root = inflater.inflate(R.layout.fragment_add_estate, container, false);

titleET    = root.findViewById(R.id.addTitleET);
descET     = root.findViewById(R.id.addDescriptionET);
cityET     = root.findViewById(R.id.addCityET);
addressET  = root.findViewById(R.id.addAddressET);
priceET    = root.findViewById(R.id.addPriceET);
areaET     = root.findViewById(R.id.addAreaET);
latET      = root.findViewById(R.id.addLatET);
lontET     = root.findViewById(R.id.addLontET);
categorySpinner = root.findViewById(R.id.addCategorySpinner);
photosCountTV  = root.findViewById(R.id.addPhotosCountTV);

Button pickB = root.findViewById(R.id.addPickPhotosB);
Button saveB = root.findViewById(R.id.addSaveB);
pickB.setOnClickListener(v -> openPicker());
saveB.setOnClickListener(v -> save());

loadCategories();
return root;
}

private void openPicker() {
    Intent i = new Intent(Intent.ACTION_GET_CONTENT);
    i.setType("image/*");
    i.putExtra(Intent.EXTRA_ALLOW_MULTIPLE, true);
    i.addCategory(Intent.CATEGORY_OPENABLE);
    pickPhotos.launch(Intent.createChooser(i, "Photos"));
}

private void loadCategories() {
    ApiClient.get("/api/categories", null, new ApiClient.Callback() {
        @Override public void onSuccess(JSONObject body) {
            try {
                categories = Category.listFromJson(body.optJSONArray("categories"));
                List<String> names = new ArrayList<>();
                for (Category c : categories) names.add(c.name);
                ArrayAdapter<String> ad = new ArrayAdapter<>(
                    requireContext(), android.R.layout.simple_spinner_dropdown_item, names);
                categorySpinner.setAdapter(ad);
            } catch (Exception ignored) { }
        }
        @Override public void onError(int httpCode, String message) {
            Toast.makeText(requireContext(), message, Toast.LENGTH_LONG).show();
        }
    });
}

private void save() {
    String title = titleET.getText().toString().trim();
    String city  = cityET.getText().toString().trim();
    String priceTxt = priceET.getText().toString().trim();
    String areaTxt  = areaET.getText().toString().trim();

    if (title.isEmpty() || city.isEmpty() || priceTxt.isEmpty() || areaTxt.isEmpty()
        || categories.isEmpty() || categorySpinner.getSelectedItemPosition() < 0) {
        Toast.makeText(requireContext(), R.string.error_message_empty_fields, Toast.LENGTH_SHORT).show();
        return;
    }

    JSONObject body = new JSONObject();
    try {
        body.put("title", title);
        body.put("description", descET.getText().toString().trim());
    }

```

```

        body.put("city", city);
        body.put("address", addressET.getText().toString().trim());
        body.put("price", Double.parseDouble(priceTxt));
        body.put("area", Double.parseDouble(areaTxt));
        body.put("categoryId", categories.get(categorySpinner.getSelectedItemPosition()).id);
        if (!latET.getText().toString().isEmpty()) body.put("lat", Double.parseDouble(latET.getText().toString()));
        if (!lonET.getText().toString().isEmpty()) body.put("lon", Double.parseDouble(lonET.getText().toString()));
    } catch (Exception e) {
        Toast.makeText(requireContext(), "Invalid number: " + e.getMessage(), Toast.LENGTH_LONG).show();
        return;
    }

    String token = new SessionManager(requireContext()).getToken();
    ApiClient.post("/api/estates", body, token, new ApiClient.Callback() {
        @Override public void onSuccess(JSONObject json) {
            JSONObject estate = json.optJSONObject("estate");
            int id = estate != null ? estate.optInt("id") : 0;
            if (id <= 0 || pickedPhotos.isEmpty()) {
                finished();
            } else {
                uploadPhotos(id);
            }
        }
        @Override public void onError(int httpCode, String message) {
            Toast.makeText(requireContext(), message, Toast.LENGTH_LONG).show();
        }
    });
}

private void uploadPhotos(int estateId) {
    List<ApiClient.FilePart> files = new ArrayList<>();
    ContentResolver cr = requireContext().getContentResolver();
    for (Uri uri : pickedPhotos) {
        try (InputStream in = cr.openInputStream(uri)) {
            if (in == null) continue;
            ByteArrayOutputStream out = new ByteArrayOutputStream();
            byte[] buf = new byte[8192];
            int n;
            while ((n = in.read(buf)) > 0) out.write(buf, 0, n);
            String mime = cr.getType(uri);
            if (mime == null) mime = "image/jpeg";
            String ext = MimeTypeMap.getSingleton().getExtensionFromMimeType(mime);
            if (ext == null) ext = "jpg";
            files.add(new ApiClient.FilePart("photos", "photo." + ext, mime, out.toByteArray()));
        } catch (Exception ignored) { }
    }
    if (files.isEmpty()) { finished(); return; }

    String token = new SessionManager(requireContext()).getToken();
    ApiClient.postMultipart("/api/estates/" + estateId + "/photos", null, files, token, new ApiClient.Callback() {
        @Override public void onSuccess(JSONObject body) { finished(); }
        @Override public void onError(int httpCode, String message) {
            Toast.makeText(requireContext(), "Photo upload failed: " + message, Toast.LENGTH_LONG).show();
            finished();
        }
    });
}

private void finished() {
    Toast.makeText(requireContext(), R.string.add_estate_saved, Toast.LENGTH_SHORT).show();
    Navigation.findNavController(requireView()).navigate(R.id.nav_estates);
}
}

```

Концепциите за студентите:

1. **ActivityResult API:**

```
private final ActivityResultLauncher<Intent> pickPhotos =  
    registerForActivityResult(new ActivityResultContracts.StartActivityForResult(), result -> { ... });
```

Регистрира се **в полето** (не в onCreate). После се ползва `pickPhotos.launch(intent)`. Заменя стария `onActivityResult(int requestCode, ...)`.

2. **Множествен избор:**

```
i.putExtra(Intent.EXTRA_ALLOW_MULTIPLE, true);
```

Резултатът се връща по два начина:

- Една снимка → `data.getData()`
- Много → `data.getClipData()`

3. **Две заявки една след друга:** backend-ът трябва да знае за кой имот да върже снимките. Първо `POST /api/estates` → получаваме `id`. После `POST /api/estates/{id}/photos`.

4. **Uri → byte[]:**

```
InputStream in = cr.openInputStream(uri);  
ByteArrayOutputStream out = new ByteArrayOutputStream();  
byte[] buf = new byte[8192];  
int n;  
while ((n = in.read(buf)) > 0) out.write(buf, 0, n);
```

`try (...)` е `try-with-resources` — `in.close()` се вика автоматично.

5. `Navigation.findNavController(requireView()).navigate(R.id.nav_estates)` — след `save` връщаме потребителя на списъка.

Стъпка 7 — Drawer меню, nav graph, MainActivity, FAB

7a. `res/menu/activity_main_drawer.xml`

Намерете:

```
<item  
    android:id="@+id/nav_slideshow"  
    android:icon="@drawable/ic_menu_slideshow"  
    android:title="@string/menu_slideshow" />
```

Заменете с:

```
<item  
    android:id="@+id/nav_add_estate"  
    android:icon="@drawable/ic_menu_slideshow"  
    android:title="@string/menu_add_estate" />
```

7b. `res/navigation/mobile_navigation.xml`

Намерете `<fragment ... nav_slideshow ... SlideshowFragment />` и заменете с:

```
<fragment  
    android:id="@+id/nav_add_estate"  
    android:name="com.example.bezbroker.ui.add.AddEstateFragment"  
    android:label="@string/menu_add_estate"  
    tools:layout="@layout/fragment_add_estate" />
```

7c. `MainActivity.java`

В `AppBarConfiguration` сменете `R.id.nav_slideshow` с `R.id.nav_add_estate`:

```
mAppBarConfiguration = new AppBarConfiguration.Builder(  
    R.id.nav_estates, R.id.nav_gallery, R.id.nav_add_estate)
```

```
.setOpenableLayout(drawer)
.build();
```

7d. FAB → отваря Add Estate

В MainActivity.onCreate намерете:

```
binding.appBarMain.fab.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        Snackbar.make(view, "Replace with your own action", Snackbar.LENGTH_LONG)
            .setAction("Action", null)
            .setAnchorView(R.id.fab).show();
    }
});
```

Заменете с:

```
binding.appBarMain.fab.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        androidx.navigation.NavController nav = androidx.navigation.Navigation.findNavController(
            MainActivity.this, R.id.nav_host_fragment_content_main);
        nav.navigate(R.id.nav_add_estate);
    }
});
```

Сега можете да премахнете import-а `import com.google.android.material.snackbar.Snackbar`; ако не се ползва другаде.

Тест

1. Run.
2. Drawer → **Add estate** (или натиснете FAB  в долния десен ъгъл).
3. Попълнете формата.
4. "Pick photos" → изберете 1-3 снимки → виждате "Pick photos: N".
5. Save → Toast "Estate saved" → връща ви на Estates → новият имот е там.
6. Натиснете на него → детайл → снимките са в карусела.

Често срещани грешки

Симптом	Причина
Picker не показва нищо	Липсва READ_MEDIA_IMAGES permission (Стъпка 1).
HTTP 415	Снимките са image/heic (от iPhone). Backend приема само jpeg/png/webp.
HTTP 413	Снимките са твърде големи (>10 MB).
App crash при getClipData	result.getData() може да е null. Проверете <code>if (data.getClipData() != null) ... else if (data.getData() != null)</code>

Следва

В **Урок 05** правим картата с osmdroid.

Урок 05 — Карта с имоти (osmdroid)

Цел

Подменяме placeholder-а GalleryFragment с MapFragment — карта на OpenStreetMap с маркер за всеки имот, който има координати. Тар върху маркер → отваря EstateDetailActivity.

Защо osmdroid вместо Google Maps?

- **Безплатно**, без API ключ.
- Без сметка в Google Cloud, без billing.
- Студентите могат да стартират проекта за 5 минути.

Какво ще променим

Действие	Файл
Добавяме	библиотека osmdroid
Добавяме	ACCESS_NETWORK_STATE permission
Преименуваме	низа menu_gallery → menu_map
Подменяме	GalleryFragment → MapFragment
Обновяваме	drawer меню, nav graph, MainActivity
Изтриваме	GalleryFragment.java, GalleryViewModel.java, fragment_gallery.xml

Estate *модела го имаме от Урок 02.*

Стъпка 1 — Библиотека osmdroid

1a. gradle/libs.versions.toml

В блока [versions] добавете:

```
osmdroid = "6.1.18"
```

В блока [libraries] добавете:

```
osmdroid = { group = "org.osmdroid", name = "osmdroid-android", version.ref = "osmdroid" }
```

1b. app/build.gradle

В dependencies блока добавете:

```
implementation libs.osmdroid
```

Натиснете "Sync Now".

Стъпка 2 — Permission

Файл: AndroidManifest.xml

Под `<uses-permission android:name="android.permission.INTERNET" />` **добавете:**

```
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

Защо: osmdroid го изисква, за да тегли картовите плочки.

Стъпка 3 — Преименуване на низа

Файл: res/values/strings.xml

Намерете:

```
<string name="menu_gallery">Gallery</string>
```

Заменете с:

```
<string name="menu_map">Map</string>
```

Стъпка 4 — Изтриване на Gallery placeholder-а

В Project панела:

- Десен бутон върху пакета `com.example.bezbroker.ui.gallery` → Delete.
- Десен бутон върху `res/layout/fragment_gallery.xml` → Delete.

Стъпка 5 — Нов пакет ui.map + layout

Десен бутон върху `com.example.bezbroker.ui` → New → Package → map.

После `res/layout` → New → Layout Resource File → име `fragment_map`, root element `FrameLayout`.

Заменете съдържанието с:

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".ui.map.MapFragment">

    <org.osmdroid.views.MapView
        android:id="@+id/mapView"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />
</FrameLayout>
```

Един `FrameLayout` с един `MapView`, който запълва целия екран. Android Studio може да показва грешка докато Gradle sync не приключи — не се тревожете.

Стъпка 6 — MapFragment.java

В `ui.map`: десен бутон → New → Java Class → `MapFragment`.

```
package com.example.bezbroker.ui.map;
```

```

import android.content.Intent;
import android.os.Bundle;
import android.preference.PreferenceManager;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;

import com.example.bezbroker.ApiClient;
import com.example.bezbroker.EstateDetailActivity;
import com.example.bezbroker.R;
import com.example.bezbroker.SessionManager;
import com.example.bezbroker.model.Estate;

import org.json.JSONObject;
import org.osmdroid.config.Configuration;
import org.osmdroid.tileprovider.tilesource.TileSourceFactory;
import org.osmdroid.util.GeoPoint;
import org.osmdroid.views.MapView;
import org.osmdroid.views.overlay.Marker;

import java.util.List;

public class MapFragment extends Fragment {

    private MapView map;

    @Override
    public View onCreateView(@NonNull LayoutInflater inflater,
                             @Nullable ViewGroup container,
                             @Nullable Bundle savedInstanceState) {
        // 1) Инициализация на osmdroid преди да inflate-нем layout-a
        Configuration.getInstance().load(
            requireContext().getApplicationContext(),
            PreferenceManager.getDefaultSharedPreferences(requireContext().getApplicationContext())
        );
        Configuration.getInstance().setUserAgentValue(requireContext().getPackageName());

        // 2) Inflate + конфигурация
        View root = inflater.inflate(R.layout.fragment_map, container, false);
        map = root.findViewById(R.id.mapView);
        map.setTileSource(TileSourceFactory.MAPNIK);
        map.setMultiTouchControls(true);
        map.getController().setZoom(6.0);
        map.getController().setCenter(new GeoPoint(42.7339, 25.4858)); // център на България

        // 3) Маркери
        loadEstates();
        return root;
    }

    @Override public void onResume() { super.onResume(); if (map != null) map.onResume(); }
    @Override public void onPause() { super.onPause(); if (map != null) map.onPause(); }

    private void loadEstates() {
        String token = new SessionManager(requireContext()).getToken();
        ApiClient.get("/api/estates", token, new ApiClient.Callback() {
            @Override public void onSuccess(JSONObject body) {
                try {
                    List<Estate> estates = Estate.listFromJson(body.optJSONArray("estates"));
                    for (Estate e : estates) {
                        if (e.lat == null || e.lont == null) continue;
                    }
                }
            }
        });
    }
}

```

```

        Marker m = new Marker(map);
        m.setPosition(new GeoPoint(e.lat, e.lont));
        m.setAnchor(Marker.ANCHOR_CENTER, Marker.ANCHOR_BOTTOM);
        m.setTitle(e.title);
        m.setSnippet(e.city + " · " + (int) e.price + " €");
        m.setOnMarkerClickListener((marker, mapView) -> {
            Intent i = new Intent(requireContext(), EstateDetailActivity.class);
            i.putExtra("estate_id", e.id);
            startActivity(i);
            return true;
        });
        map.getOverlays().add(m);
    }
    map.invalidate();
} catch (Exception ignored) { }
}
@Override public void onError(int httpCode, String message) { }
});
}
}

```

Концепциите за студентите:

1. Инициализация преди inflate

```

Configuration.getInstance().load(applicationContext, sharedPreferences);
Configuration.getInstance().setUserAgentValue(packageName);

```

osmdroid пази настройки в SharedPreferences. Тези две линии **трябва** да са преди `inflater.inflate(...)`, иначе ще получите `NullPointerException` или сива карта.

`setUserAgentValue` е критичен — OpenStreetMap сървърите могат да блокират анонимни заявки.

2. Tile source

```
map.setTileSource(TileSourceFactory.MAPNIK);
```

MAPNIK е default стилът на OpenStreetMap.

3. Multi-touch + zoom + center

```

map.setMultiTouchControls(true);           // pinch-to-zoom
map.getController().setZoom(6.0);
map.getController().setCenter(new GeoPoint(42.7339, 25.4858));

```

`GeoPoint(lat, lon)` — географски координати. 42.7339, 25.4858 е геометричният център на България.

4. Lifecycle

```

@Override public void onResume() { ... map.onResume(); }
@Override public void onPause() { ... map.onPause(); }

```

Без `onPause` картата продължава да тегли плочки на background — губим батерия и трафик.

5. Marker

```

Marker m = new Marker(map);
m.setPosition(new GeoPoint(e.lat, e.lont));
m.setAnchor(Marker.ANCHOR_CENTER, Marker.ANCHOR_BOTTOM);
m.setOnMarkerClickListener(...);
map.getOverlays().add(m);

```

`setAnchor(CENTER, BOTTOM)` — върхът на "иглата" сочи точно координатата.

6. return true в click listener-a

```
m.setOnMarkerClickListener((marker, mapView) -> {  
    startActivity(intent);  
    return true;  
});
```

true казва на osmdroid "аз се справих с tap-a; не показвай default info window-то".

7. Защо проверяваме lat == null

Не всеки имот има координати — потребителят може да е пропуснал полетата при Add. Пропускаме такива.

Стъпка 7 — Drawer меню, nav graph, MainActivity

7a. res/menu/activity_main_drawer.xml

Намерете <item ... nav_gallery ... /> и заменете с:

```
<item  
    android:id="@+id/nav_map"  
    android:icon="@drawable/ic_menu_gallery"  
    android:title="@string/menu_map" />
```

7b. res/navigation/mobile_navigation.xml

Намерете <fragment ... nav_gallery ... GalleryFragment /> и заменете с:

```
<fragment  
    android:id="@+id/nav_map"  
    android:name="com.example.bezbroker.ui.map.MapFragment"  
    android:label="@string/menu_map"  
    tools:layout="@layout/fragment_map" />
```

7c. MainActivity.java

В AppBarConfiguration сменете R.id.nav_gallery с R.id.nav_map:

```
mAppBarConfiguration = new AppBarConfiguration.Builder(  
    R.id.nav_estates, R.id.nav_map, R.id.nav_add_estate)  
    .setOpenableLayout(drawer)  
    .build();
```

Тест

1. Стартирайте приложението.
2. Drawer → **Map**.
3. Виждате карта на България (zoom 6 показва цялата страна).
4. Pinch-to-zoom работи.
5. Ако сте добавили имот с координати (от Урок 04, например lat=42.6977, lont=23.3219 за Sofia) → маркер.
6. Tap на маркер → отваря Detail.

Често срещани грешки

Симптом	Причина
Сива карта без плочки	Липсва <code>INTERNET</code> или <code>ACCESS_NETWORK_STATE</code> .
<code>Tile fetching failed</code> в Logcat	<code>setUserAgentValue</code> не е извикан.
Картата flicker-ва или тече памет	Не сте свързали <code>onResume/onPause</code> .
<code>ClassNotFoundException</code> за <code>Configuration</code>	Грешен <code>import</code> . Трябва <code>org.osmdroid.config.Configuration</code> , не <code>android.content.res.Configuration</code> .

Следва

В **Урок 06** правим `OffersFragment` — четвърти елемент в `drawer-a`.

Урок 06 — Оферти

Цел

Добавяме **четвърти** елемент в drawer-а (нова дестинация, не подмяна) — OffersFragment, който показва:

- офертите, които сме направили (като купувач);
- офертите, които сме получили (като продавач).

За всяка оферта: Асепт / Reject (само ако сме продавачът и state е още pending) + Chat (винаги). Chat-ът засега ще показва Toast — в Урок 07 ще го свържем с ChatActivity.

Какво ще променим

Действие	Файл
Добавяме	низове за offers
Добавяме	модел Offer
Добавяме	OfferAdapter + item_offer.xml
Създаваме	OffersFragment + fragment_offers.xml
Обновяваме	drawer меню (нов елемент), nav graph (нов destination), MainActivity

RecyclerView вече сме го добавили в Урок 02. Никакви нови библиотеки или permissions в този урок.

Стъпка 1 — Низове

Файл: res/values/strings.xml

Преди </resources> **добавете:**

```
<!-- Offers -->
<string name="menu_offers">Offers</string>
<string name="no_offers">No offers yet</string>
<string name="accept">Accept</string>
<string name="reject">Reject</string>
<string name="chat">Chat</string>
```

Стъпка 2 — Моделът Offer

Файл: model/Offer.java (нов)

```
package com.example.bezbroker.model;

import org.json.JSONArray;
import org.json.JSONException;
import org.json.JSONObject;

import java.util.ArrayList;
import java.util.List;

public class Offer {
```

```

public int id;
public double sum;
public int offeredByUser;
public int offeredForEstate;
public String state;           // "pending" | "accepted" | "rejected"
public String createdAt;
public String estateTitle;
public int estateOwnerId;
public String buyerName;
public String sellerName;

public static Offer fromJson(JSONObject o) throws JSONException {
    Offer x = new Offer();
    x.id          = o.optInt("id");
    x.sum         = o.optDouble("sum", 0);
    x.offeredByUser = o.optInt("offeredByUser");
    x.offeredForEstate = o.optInt("offeredForEstate");
    x.state       = o.optString("state", "pending");
    x.createdAt   = o.optString("createdAt", "");
    x.estateTitle = o.optString("estateTitle", "");
    x.estateOwnerId = o.optInt("estateOwnerId");
    x.buyerName   = o.optString("buyerName", "");
    x.sellerName  = o.optString("sellerName", "");
    return x;
}

public static List<Offer> listFromJson(JSONArray arr) throws JSONException {
    List<Offer> out = new ArrayList<>();
    if (arr == null) return out;
    for (int i = 0; i < arr.length(); i++) out.add(fromJson(arr.getJSONObject(i)));
    return out;
}
}

```

estateTitle, estateOwnerId, buyerName, sellerName са "join" полета — backend-ът ги прави от свързаните таблици и ни ги дава наготово, за да не правим още заявки.

Стъпка 3 — item_offer.xml

res/layout → New → Layout Resource File → име item_offer, root element com.google.android.material.card.MaterialCardView.

Заменете съдържанието с:

```

<?xml version="1.0" encoding="utf-8"?>
<com.google.android.material.card.MaterialCardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="8dp"
    app:cardCornerRadius="8dp"
    app:cardElevation="2dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="12dp">

        <TextView
            android:id="@+id/itemOfferTitle"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"

```

```

        android:textSize="16sp"
        android:textStyle="bold" />

<TextView
    android:id="@+id/itemOfferSum"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textColor="#2941CE" />

<TextView
    android:id="@+id/itemOfferMeta"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textColor="#666666"
    android:textSize="12sp" />

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:orientation="horizontal">

    <Button
        android:id="@+id/itemOfferAcceptB"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="@string/accept" />

    <Button
        android:id="@+id/itemOfferRejectB"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginStart="4dp"
        android:layout_weight="1"
        android:text="@string/reject" />

    <Button
        android:id="@+id/itemOfferChatB"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginStart="4dp"
        android:layout_weight="1"
        android:text="@string/chat" />
    </LinearLayout>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

```

Стъпка 4 — OfferAdapter

Файл: adapter/OfferAdapter.java (в пакета adapter, който вече съществува от Урок 02)

```

package com.example.bezbroker.adapter;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.example.bezbroker.R;

```

```

import com.example.bezbroker.model.Offer;

import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

public class OfferAdapter extends RecyclerView.Adapter<OfferAdapter.VH> {

    public interface Listener {
        void onAccept(Offer offer);
        void onReject(Offer offer);
        void onChat(Offer offer);
    }

    private final List<Offer> items = new ArrayList<>();
    private final Listener listener;
    private final int currentUserId;

    public OfferAdapter(int currentUserId, Listener listener) {
        this.currentUserId = currentUserId;
        this.listener = listener;
    }

    public void setItems(List<Offer> list) {
        items.clear();
        items.addAll(list);
        notifyDataSetChanged();
    }

    @NonNull @Override
    public VH onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
        View v = LayoutInflater.from(parent.getContext()).inflate(R.layout.item_offer, parent, false);
        return new VH(v);
    }

    @Override
    public void onBindViewHolder(@NonNull VH h, int position) {
        Offer o = items.get(position);
        h.title.setText(o.estateTitle);
        h.sum.setText(String.format(Locale.getDefault(), "%.0f € · %s", o.sum, o.state));

        boolean iAmSeller = o.estateOwnerId == currentUserId;
        String partner = iAmSeller
            ? "From: " + (o.buyerName.isEmpty() ? "(buyer)" : o.buyerName)
            : "To: " + (o.sellerName.isEmpty() ? "(seller)" : o.sellerName);
        h.meta.setText(partner + " · " + o.createdAt);

        boolean canDecide = iAmSeller && "pending".equals(o.state);
        h.accept.setVisibility(canDecide ? View.VISIBLE : View.GONE);
        h.reject.setVisibility(canDecide ? View.VISIBLE : View.GONE);

        h.accept.setOnClickListener(v -> listener.onAccept(o));
        h.reject.setOnClickListener(v -> listener.onReject(o));
        h.chat.setOnClickListener(v -> listener.onChat(o));
    }

    @Override public int getItemCount() { return items.size(); }

    static class VH extends RecyclerView.ViewHolder {
        TextView title, sum, meta;
        Button accept, reject, chat;
        VH(@NonNull View v) {
            super(v);
            title = v.findViewById(R.id.itemOfferTitle);
            sum = v.findViewById(R.id.itemOfferSum);
            meta = v.findViewById(R.id.itemOfferMeta);
        }
    }
}

```

```

        accept = v.findViewById(R.id.itemOfferAcceptB);
        reject = v.findViewById(R.id.itemOfferRejectB);
        chat    = v.findViewById(R.id.itemOfferChatB);
    }
}
}

```

Логиката на бутоните:

```

boolean iAmSeller = o.estateOwnerId == currentUserId;
boolean canDecide = iAmSeller && "pending".equals(o.state);

```

- Купувач (не съм собственик): никога не виждам Accept/Reject — продавачът решава.
- Продавач + офертата е още pending: виждам Accept/Reject.
- Продавач + офертата вече е accepted/rejected: бутоните изчезват.
- Chat: винаги достъпен.

Стъпка 5 — Нов пакет ui.offers + layout

Десен бутон върху com.example.bezbroker.ui → New → Package → offers.

res/layout → New → Layout Resource File → име fragment_offers, root element FrameLayout.

```

<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".ui.offers.OffersFragment">

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/offersRV"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:padding="8dp" />

    <TextView
        android:id="@+id/offersEmptyTV"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:text="@string/no_offers"
        android:textColor="#666666"
        android:visibility="gone" />
</FrameLayout>

```

Защо FrameLayout: искаме RecyclerView да запълни целия екран, и върху него да има центриран текст "No offers yet". FrameLayout слага дъщерните view-ове един върху друг.

Стъпка 6 — OffersFragment.java

В ui.offers: десен бутон → New → Java Class → OffersFragment.

```

package com.example.bezbroker.ui.offers;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;
import android.widget.Toast;

```

```

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.example.bezbroker.ApiClient;
import com.example.bezbroker.R;
import com.example.bezbroker.SessionManager;
import com.example.bezbroker.adapter.OfferAdapter;
import com.example.bezbroker.model.Offer;

import org.json.JSONObject;

import java.util.List;

public class OffersFragment extends Fragment {

    private OfferAdapter adapter;
    private TextView emptyTV;

    @Override
    public View onCreateView(@NonNull LayoutInflater inflater,
                             @Nullable ViewGroup container,
                             @Nullable Bundle savedInstanceState) {
        View root = inflater.inflate(R.layout.fragment_offers, container, false);

        emptyTV = root.findViewById(R.id.offersEmptyTV);
        RecyclerView rv = root.findViewById(R.id.offersRV);
        rv.setLayoutManager(new LinearLayoutManager(requireContext()));

        int myId = currentUserid();
        adapter = new OfferAdapter(myId, new OfferAdapter.Listener() {
            @Override public void onAccept(Offer o) { changeState(o, "accepted"); }
            @Override public void onReject(Offer o) { changeState(o, "rejected"); }
            @Override public void onChat(Offer o) {
                // Placeholder – в Урок 07 ще отворим ChatActivity
                Toast.makeText(requireContext(),
                    "Chat for offer #" + o.id, Toast.LENGTH_SHORT).show();
            }
        });
        rv.setAdapter(adapter);
        return root;
    }

    @Override
    public void onResume() {
        super.onResume();
        load();
    }

    private void load() {
        String token = new SessionManager(requireContext()).getToken();
        ApiClient.get("/api/offers", token, new ApiClient.Callback() {
            @Override public void onSuccess(JSONObject body) {
                try {
                    List<Offer> offers = Offer.listFromJson(body.optJSONArray("offers"));
                    emptyTV.setVisibility(offers.isEmpty() ? View.VISIBLE : View.GONE);
                    adapter.setItems(offers);
                } catch (Exception e) {
                    Toast.makeText(requireContext(), e.getMessage(), Toast.LENGTH_LONG).show();
                }
            }
            @Override public void onError(int httpCode, String message) {
                Toast.makeText(requireContext(), message, Toast.LENGTH_LONG).show();
            }
        })
    }
}

```

```

    });
}

private void changeState(Offer o, String newState) {
    JSONObject body = new JSONObject();
    try { body.put("state", newState); } catch (Exception ignored) { }
    String token = new SessionManager(requireContext()).getToken();
    ApiClient.post("/api/offers/" + o.id + "/state", body, token, new ApiClient.Callback() {
        @Override public void onSuccess(JSONObject body) { load(); }
        @Override public void onError(int httpCode, String message) {
            Toast.makeText(requireContext(), message, Toast.LENGTH_LONG).show();
        }
    });
}

private int currentUserId() {
    try {
        String json = new SessionManager(requireContext()).getUserJson();
        if (json == null) return 0;
        return new JSONObject(json).optInt("id", 0);
    } catch (Exception e) { return 0; }
}
}

```

Какво се случва:

- `onResume` → `load()` презарежда ofertите всеки път при отваряне.
- При `Accept/Reject` → `POST /api/offers/{id}/state` → `load()` за свежи данни.
- `Chat` бутонът засега показва `Toast` — Урок 07 ще го свърже с `ChatActivity`.

Стъпка 7 — Drawer меню (нов елемент)

Файл: `res/menu/activity_main_drawer.xml`

Това е **нов** елемент — не подмяна. В `<group>`-а, под последния `<item>` (`Add estate`), **добавете**:

```

<item
    android:id="@+id/nav_offers"
    android:icon="@drawable/ic_menu_camera"
    android:title="@string/menu_offers" />

```

Стъпка 8 — Nav graph (нов destination)

Файл: `res/navigation/mobile_navigation.xml`

Преди затварящата `</navigation>` тага **добавете**:

```

<fragment
    android:id="@+id/nav_offers"
    android:name="com.example.bezbroker.ui.offers.OffersFragment"
    android:label="@string/menu_offers"
    tools:layout="@layout/fragment_offers" />

```

Стъпка 9 — MainActivity

В `AppBarConfiguration` **добавете** `R.id.nav_offers`:

```

mAppBarConfiguration = new AppBarConfiguration.Builder(
    R.id.nav_estates, R.id.nav_map, R.id.nav_add_estate, R.id.nav_offers)
    .setOpenableLayout(drawer)

```

```
.build();
```

Тест на пълния поток (с 2 потребителя)

С 2 акаунта (можете да ползвате два emulator-a):

1. **A** (seller): регистрира се с Seller ☐, добавя имот.
2. **B** (buyer): регистрира се в друг emulator, Estates → клик на имота → Make offer 80000.
3. **A**: drawer → **Offers** → вижда офертата. Натиска **Accept**.
4. Списъкът се презарежда → state-ът показва "accepted", бутоните Accept/Reject изчезват.

Chat бутонът засега показва Toast.

Често срещани грешки

Симптом	Причина
Празен списък	Все още няма направени оферти. От втори акаунт направете оферта.
Асепт върнал 403	Само собственикът на имота може да приема. Проверете акаунта.
Бутоните не изчезват след асепт	load() не е извикан в callback-а на changeState.
0 като currentUserId	SessionManager.getUserJson() връща null.

Следва

В **Урок 07** правим ChatActivity и заменяме Toast-а в Chat бутона.

Урок 07 — Чат

Цел

Когато потребителят натисне Chat върху оферта от OffersFragment → отваряме отделен екран:

- горе: списък със съобщения, моите вдясно (синьо), на другия вляво (сиво);
- долу: поле за писане + Send бутон;
- автоматично обновяване на всеки 3 секунди (polling).

Защо polling, а не WebSocket?

- WebSocket изисква постоянно отворена връзка — много по-сложно.
- Polling е просто "вика API-то периодически". Лесно за обяснение, работи на всеки backend.
- Backend-ът поддържа параметър ?since=N — връща само съобщения с id > N, така не дърпаме целия чат всеки път.

Какво ще променим

Действие	Файл
Добавяме	низове за chat
Добавяме	ChatActivity в манифеста
Добавяме	модел Message
Добавяме	MessageAdapter + item_message.xml
Създаваме	activity_chat.xml
Създаваме	ChatActivity.java
Обновяваме	Chat бутона в OffersFragment (Toast → startActivity)

Никакви нови библиотеки или permissions в този урок.

Стъпка 1 — Низове

Файл: res/values/strings.xml

Преди </resources> **добавете**:

```
<!-- Chat -->
<string name="chat_input_hint">Type a message...</string>
<string name="send">Send</string>
```

chat низа го имаме от Урок 06.

Стъпка 2 — Регистриране в манифеста

Файл: AndroidManifest.xml

В <application> блока, до другите <activity> декларации, **добавете**:

```
<activity
    android:name=".ChatActivity"
    android:exported="false" />
```

Стъпка 3 — Моделът Message

Файл: model/Message.java (нов)

```
package com.example.bezbroker.model;

import org.json.JSONArray;
import org.json.JSONException;
import org.json.JSONObject;

import java.util.ArrayList;
import java.util.List;

public class Message {
    public int id;
    public String text;
    public int fromUser;
    public int toUser;
    public int offerId;
    public String createdAt;

    public static Message fromJson(JSONObject o) throws JSONException {
        Message m = new Message();
        m.id = o.optInt("id");
        m.text = o.optString("text", "");
        m.fromUser = o.optInt("fromUser");
        m.toUser = o.optInt("toUser");
        m.offerId = o.optInt("offerId");
        m.createdAt = o.optString("createdAt", "");
        return m;
    }

    public static List<Message> listFromJson(JSONArray arr) throws JSONException {
        List<Message> out = new ArrayList<>();
        if (arr == null) return out;
        for (int i = 0; i < arr.length(); i++) out.add(fromJson(arr.getJSONObject(i)));
        return out;
    }
}
```

fromUser ще ползваме в адаптера — според това дали fromUser == currentUserId, ще подравнявам балона надясно или наляво.

Стъпка 4 — item_message.xml

res/layout → New → Layout Resource File → име item_message, root element FrameLayout.

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginVertical="4dp">

    <TextView
        android:id="@+id/itemMessageTV"
        android:layout_width="wrap_content"
```

```

        android:layout_height="wrap_content"
        android:background="#EEEEEE"
        android:maxLength="260dp"
        android:padding="10dp"
        android:textColor="#000000" />
    </FrameLayout>

```

FrameLayout запълва ширината. TextView е с wrap_content — широк е колкото текста. В адаптера сменяме layout_gravity на START или END за ляво/дясно поместване.

maxLength="260dp" — за да не може дълъг текст да заеме цялата ширина.

Стъпка 5 — MessageAdapter

Файл: adapter/MessageAdapter.java

```

package com.example.bezbroker.adapter;

import android.view.Gravity;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.FrameLayout;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.example.bezbroker.R;
import com.example.bezbroker.model.Message;

import java.util.ArrayList;
import java.util.List;

public class MessageAdapter extends RecyclerView.Adapter<MessageAdapter.VH> {

    private final List<Message> items = new ArrayList<>();
    private final int currentUserId;

    public MessageAdapter(int currentUserId) {
        this.currentUserId = currentUserId;
    }

    public void appendAll(List<Message> more) {
        int start = items.size();
        items.addAll(more);
        notifyItemRangeInserted(start, more.size());
    }

    public int lastId() { return items.isEmpty() ? 0 : items.get(items.size() - 1).id; }

    @NonNull @Override
    public VH onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
        View v = LayoutInflater.from(parent.getContext()).inflate(R.layout.item_message, parent, false);
        return new VH(v);
    }

    @Override
    public void onBindViewHolder(@NonNull VH h, int position) {
        Message m = items.get(position);
        h.tv.setText(m.text);

        FrameLayout.LayoutParams lp = (FrameLayout.LayoutParams) h.tv.getLayoutParams();
        if (m.fromUser == currentUserId) {

```

```

        lp.gravity = Gravity.END;
        h.tv.setBackgroundColor(0xFFCDE4FF);
    } else {
        lp.gravity = Gravity.START;
        h.tv.setBackgroundColor(0xFFEEEEEE);
    }
    h.tv.setLayoutParams(lp);
}

@Override public int getItemCount() { return items.size(); }

static class VH extends RecyclerView.ViewHolder {
    TextView tv;
    VH(@NonNull View v) {
        super(v);
        tv = v.findViewById(R.id.itemMessageTV);
    }
}
}

```

Какво да обясним на студентите:

1. **addAll** **ВМЕСТО** **setItems**: при polling добавяме нови съобщения, не презаписваме списъка. `notifyItemRangeInserted(start, count)` казва на RecyclerView "тези редове са нови" → плавна анимация.
2. `lastId()` — най-голямото id в локалния списък. Подаваме го като `since` параметър.
3. **Left vs right c Gravity**: `FrameLayout` поддържа `layout_gravity` за всеки от дъщерните си view-ове. `lp.gravity = Gravity.END` → вдясно. `Gravity.START` → вляво.

Стъпка 6 — activity_chat.xml

res/layout → New → Layout Resource File → име activity_chat, root element LinearLayout.

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/chatRV"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:padding="8dp" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:padding="8dp">

        <EditText
            android:id="@+id/chatInputET"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:hint="@string/chat_input_hint"
            android:inputType="text" />

        <Button
            android:id="@+id/chatSendB"
            android:layout_width="wrap_content"

```

```

        android:layout_height="wrap_content"
        android:layout_marginStart="8dp"
        android:text="@string/send" />
    </LinearLayout>
</LinearLayout>

```

`layout_height="0dp" + layout_weight="1"` за RecyclerView — стандартен Android trick: запълва **ВСИЧКОТО** останало вертикално пространство.

Стъпка 7 — ChatActivity.java

В пакета `com.example.bezbroker`: New → Java Class → ChatActivity.

```

package com.example.bezbroker;

import android.os.Bundle;
import android.os.Handler;
import android.os.Looper;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.example.bezbroker.adapter.MessageAdapter;
import com.example.bezbroker.model.Message;

import org.json.JSONObject;

import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class ChatActivity extends AppCompatActivity {

    private static final long POLL_MS = 3000;

    private int offerId;
    private MessageAdapter adapter;
    private RecyclerView rv;
    private EditText input;
    private final Handler handler = new Handler(Looper.getMainLooper());
    private final Runnable pollTask = new Runnable() {
        @Override public void run() {
            pollOnce();
            handler.postDelayed(this, POLL_MS);
        }
    };

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_chat);

        offerId = getIntent().getIntExtra("offer_id", 0);
        if (offerId == 0) { finish(); return; }
        setTitle(getIntent().getStringExtra("offer_title"));

        rv = findViewById(R.id.chatRV);
        rv.setLayoutManager(new LinearLayoutManager(this));
        adapter = new MessageAdapter(currentUserId());
        rv.setAdapter(adapter);
    }
}

```

```

        input = findViewById(R.id.chatInputET);
        Button send = findViewById(R.id.chatSendB);
        send.setOnClickListener(v -> sendMessage());
    }

    @Override protected void onResume() {
        super.onResume();
        handler.post(pollTask);
    }

    @Override protected void onPause() {
        super.onPause();
        handler.removeCallbacks(pollTask);
    }

    private void pollOnce() {
        Map<String, String> q = new HashMap<>();
        q.put("since", String.valueOf(adapter.lastId()));
        String token = new SessionManager(this).getToken();
        ApiClient.get("/api/offers/" + offerId + "/messages", q, token, new ApiClient.Callback() {
            @Override public void onSuccess(JSONObject body) {
                try {
                    List<Message> more = Message.listFromJson(body.optJSONArray("messages"));
                    if (!more.isEmpty()) {
                        adapter.appendAll(more);
                        rv.scrollToPosition(adapter.getItemCount() - 1);
                    }
                } catch (Exception ignored) { }
            }
        });
        @Override public void onError(int httpCode, String message) { /* keep polling */ }
    });
}

private void sendMessage() {
    String text = input.getText().toString().trim();
    if (text.isEmpty()) return;
    JSONObject body = new JSONObject();
    try { body.put("text", text); } catch (Exception ignored) { }
    input.setText("");
    String token = new SessionManager(this).getToken();
    ApiClient.post("/api/offers/" + offerId + "/messages", body, token, new ApiClient.Callback() {
        @Override public void onSuccess(JSONObject body) { pollOnce(); }
        @Override public void onError(int httpCode, String message) {
            Toast.makeText(ChatActivity.this, message, Toast.LENGTH_LONG).show();
        }
    });
}

private int currentUserId() {
    try {
        String json = new SessionManager(this).getUserJson();
        if (json == null) return 0;
        return new JSONObject(json).optInt("id", 0);
    } catch (Exception e) { return 0; }
}
}

```

Концепциите за студентите:

1. Polling с Handler и Runnable

```
private final Handler handler = new Handler(Looper.getMainLooper());
private final Runnable pollTask = new Runnable() {
    @Override public void run() {
        pollOnce();
        handler.postDelayed(this, POLL_MS);
    }
};
```

Handler изпълнява код на UI thread след определено време. Класическият патерн за повторение е:

1. Runnable.run() прави работата.
2. В края postDelayed(this, X) се планира run() отново след X милисекунди.

2. Стартиране и спиране в lifecycle

```
@Override protected void onResume() { handler.post(pollTask); }
@Override protected void onPause() { handler.removeCallbacks(pollTask); }
```

- Екранът става видим → пускаме polling.
- Потребителят излезе → спираме.

removeCallbacks(pollTask) казва "забрави всички планирани извиквания". Без него телефонът пали трафик дори на background.

3. Параметър since

- При първи poll: lastId() == 0 → сървърът връща **всички** съобщения.
- При следващи: lastId е id-то на последното → връща само по-нови.

4. Auto-scroll до дъното

```
adapter.appendAll(more);
rv.scrollToPosition(adapter.getItemCount() - 1);
```

Потребителят винаги вижда последното съобщение без да трябва да скролва ръчно.

5. Оптимистично изтриване на input-a

```
input.setText("");
ApiClient.post(...);
```

Изтриваме **преди** API-то да отговори — потребителят усеща приложението като отзивчиво. При успех pollOnce() ще донесе и моето собствено съобщение.

Стъпка 8 — Свързване от OffersFragment

Файл: ui/offers/OffersFragment.java

В onCreateView намерете click handler-a от Урок 06:

```
@Override public void onChat(Offer o) {
    // Placeholder — в Урок 07 ще отворим ChatActivity
    Toast.makeText(requireContext(),
        "Chat for offer #" + o.id, Toast.LENGTH_SHORT).show();
}
```

Заменете с:

```
@Override public void onChat(Offer o) {
    Intent i = new Intent(requireContext(), ChatActivity.class);
    i.putExtra("offer_id", o.id);
    i.putExtra("offer_title", o.estateTitle);
}
```

```
        startActivity(i);  
    }
```

Добавете import-ите:

```
import android.content.Intent;  
import com.example.bezbroker.ChatActivity;
```

Ако Toast вече не се ползва другаде в `onChat`, можете да премахнете `import android.widget.Toast`; — но Toast се ползва и в `onError` callback-ите, така че остава.

Тест на пълния поток

С 2 акаунта / 2 emulator-a:

1. **A** (seller): Add estate → Save.
2. **B** (buyer): Estates → клик → Make offer 80000.
3. **A**: Offers → клик **Chat** върху офертата.
4. Чат екранът се отваря (празен).
5. A пише "Може ли да дойдете в петък?" → Send. Балонът се появява вдясно.
6. **B**: Offers → Chat → след ≤3 секунди вижда съобщението вляво (от друг потребител).
7. B пише "Да, в 17:00" → Send.
8. След ≤3 секунди A вижда отговора.

Често срещани грешки

Симптом	Причина
Съобщенията не пристигат от другата страна	Polling не е стартиран. Проверете <code>onResume</code> → <code>handler.post(pollTask)</code> .
Дублирани съобщения	Не пращате <code>since</code> или <code>lastId()</code> е грешен.
Балоните винаги отляво	<code>currentUserId()</code> връща 0. Проверете <code>SessionManager</code> .
Polling продължава след излизане	<code>onPause</code> не вика <code>removeCallbacks</code> . В Logcat ще видите периодични мрежови заявки.

Следва

В **Урок 08** добавяме Logout — последен щрих.

Урок 08 — Logout

Цел

Последен щрих. Добавяме Logout като отделен елемент в drawer-а — извън главната група, защото не е "destination" (не е свързан с фрагмент), а действие.

При натискане:

1. Изтрива се сесията (`SessionManager.clear()`).
2. Отваря се `LoginActivity` като start activity.
3. Затваря се `MainActivity`.

Какво ще променим

Действие	Файл
Добавяме	низ <code>menu_logout</code>
Обновяваме	drawer меню — нов елемент извън group-а
Обновяваме	<code>MainActivity</code> — прихващаме клика върху Logout

Никакви нови библиотеки, permissions или модели.

Стъпка 1 — Низ

Файл: `res/values/strings.xml`

Преди `</resources>` **добавете:**

```
<string name="menu_logout">Logout</string>
```

Стъпка 2 — Drawer меню (нов елемент извън group-а)

Файл: `res/menu/activity_main_drawer.xml`

Текущата структура (след предишните уроци) изглежда така:

```
<menu ...>
  <group android:checkableBehavior="single">
    <item android:id="@+id/nav_estates" ... />
    <item android:id="@+id/nav_map" ... />
    <item android:id="@+id/nav_add_estate" ... />
    <item android:id="@+id/nav_offers" ... />
  </group>
</menu>
```

Извън <group>-а (но преди затварящия `</menu>`) **добавете:**

```
<item
  android:id="@+id/nav_logout"
  android:title="@string/menu_logout" />
```

Защо извън group-а: group-ът с `checkableBehavior="single"` означава "това са взаимно изключващи се destination-и; точно един може да е селектиран". Logout не е destination — той е действие. Слагайки го

извън group-а:

- Не получава галка/радио бутон.
- Не се "избира" — само се натиска.
- Появява се с разделителна линия над него (стандартен Material drawer pattern).

Стъпка 3 — Прихващане на клика в MainActivity

Файл: MainActivity.java

Сега drawer-ът има 4 destination-а (които Navigation Component обработва автоматично) + 1 действие (Logout, което трябва **ние** да обработим).

Намерете в onCreate тази линия:

```
NavigationUI.setupWithNavController(navigationView, navController);
```

Под нея добавете:

```
// Logout не е destination – прихващаме го ръчно
navigationView.setNavigationItemSelectedListener(item -> {
    if (item.getItemId() == R.id.nav_logout) {
        new SessionManager(MainActivity.this).clear();
        Intent i = new Intent(MainActivity.this, LoginActivity.class);
        i.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TASK | Intent.FLAG_ACTIVITY_NEW_TASK);
        startActivity(i);
        finish();
        return true;
    }
    boolean handled = NavigationUI.onNavDestinationSelected(item, navController);
    if (handled) drawer.close();
    return handled;
});
```

Добавете import-а в горната част на файла, ако го няма:

```
import android.content.Intent;
```

Какво се случва, ред по ред:

Ред	Какво прави
<code>if (item.getItemId() == R.id.nav_logout)</code>	Прихващаме само Logout; всички останали ги оставяме за NavigationUI.
<code>new SessionManager(this).clear()</code>	Изтрива токена и user JSON от SharedPreferences.
<code>Intent.FLAG_ACTIVITY_CLEAR_TASK + NEW_TASK</code>	Затваря всички activity-та в стека и стартира LoginActivity като нова task. Така потребителят не може с Back да се върне в логнатия екран.
<code>finish()</code>	Затваря MainActivity.
<code>return true</code>	Казва на NavigationView "аз се справих с клика".
<code>NavigationUI.onNavDestinationSelected(item, navController)</code>	За всички НЕ-logout кликове — стандартното навигиране (отвори съответния фрагмент).
<code>if (handled) drawer.close()</code>	Затваря drawer-а след избор (NavigationUI не го прави автоматично при ръчно прихващане).

Важно: `setNavigationItemSelectedListener` *презаписва това, което* `setupWithNavController(navigationView, navController)` *направи в предишния ред. Затова в нашия handler трябва да викнем* `NavigationUI.onNavDestinationSelected(...)` *за всички HE-logout случаи — иначе обикновените destination-и спират да работят.*

Тест

1. Run.
2. Логнете се (ако вече не сте).
3. Отворете drawer-а — виждате:
 - Estates
 - Map
 - Add estate
 - Offers
 - ————— (разделителна линия)
 - **Logout**
4. Натиснете **Logout**.
5. Връщате се на Login екрана.
6. Натиснете системния Back бутон → приложението излиза (не се връщате в MainActivity).
7. Login пак → попаднете обратно в Estates.

Често срещани грешки

Симптом	Причина
След Logout натискате Back и се връщате в MainActivity	Пропуснали сте <code>FLAG_ACTIVITY_CLEAR_TASK</code> .
Кликовете върху Estates/Map/... спират да работят след добавянето на listener-а	Забравили сте <code>NavigationUI.onNavDestinationSelected(item, navController)</code> в <code>else branch</code> -а.
Drawer-ът не се затваря след навигация	Липсва <code>drawer.close()</code> след <code>handled</code> .
Logout прави нищо	Пропуснали сте <code>nav_logout</code> в drawer XML.

Поздравления — приложението е готово!

Финална структура на проекта:

```
com.example.bezbroker/  
├─ ApiClient.java      (разширихме с get+query, postMultipart, FilePart – Уроци 02, 04)  
├─ SessionManager.java (не сме пипали)  
├─ LoginActivity.java  (не сме пипали)  
├─ RegisterActivity.java (не сме пипали)  
├─ MainActivity.java    (Уроци 02-08: drawer rewiring, FAB, Logout)  
├─ EstateDetailActivity.java (Урок 03)  
├─ ChatActivity.java    (Урок 07)  
├─ adapter/  
│   ├─ EstateAdapter.java (Урок 02)  
│   └─ OfferAdapter.java  (Урок 06)
```

```

├── MessageAdapter.java    (Урок 07)
├── model/
│   ├── Estate.java       (Урок 02)
│   ├── Category.java     (Урок 02)
│   ├── Offer.java        (Урок 06)
│   └── Message.java      (Урок 07)
├── ui/
│   ├── estates/EstatesFragment.java (Урок 02, click handler ъпдейтнат в Урок 03)
│   ├── map/MapFragment.java (Урок 05)
│   ├── add/AddEstateFragment.java (Урок 04)
│   └── offers/OffersFragment.java (Урок 06, chat handler ъпдейтнат в Урок 07)

```

Drawer:

```

BezBroker
• Estates      ← Урок 02
• Map          ← Урок 05
• Add estate   ← Урок 04
• Offers       ← Урок 06
└──
• Logout      ← Урок 08

```

Финална проверка (с 2 акаунта)

Стъпка	Потребител А (seller)	Потребител В (buyer)
1	Register (Seller ☐)	Register
2	Add estate → форма + 2 снимки → Save	—
3	—	Estates → вижда новия имот → клик
4	—	Detail → Make offer 80000
5	Offers → вижда офертата на В	—
6	Натиска Chat → "Здравейте" → Send	—
7	—	Offers → Chat → след ≤3 сек вижда "Здравейте"
8	—	"Кога мога да го видя?" → Send
9	След ≤3 сек вижда отговора	—
10	Offers → Accept	—
11	—	Offers → state-ът е "accepted"
12	Map → вижда маркер	—
13	Logout → връща се на Login	—

Ако всичко минава без crash и без грешки в Logcat — успех!

Идеи за следващи часове

- **Push notifications** (Firebase Cloud Messaging) вместо polling.
- **Запазване на търсене** в локална SQLite (Room).

- **Карта с tap-to-pin** при добавяне на имот (вместо ръчно въвеждане на координати).
- **Admin** роля за модериране на обяви.
- **Pagination** в Estates списъка (?offset=N&limit=20).